



Name of the module: TX IQ compensation

Location in GIT repository: vspa_common\vspa-lib\txiqcomp

Type of code:

- 1 VCPU kernel function

VCPU Function API:

```
extern void txiqcomp_x32chf_5t( vspa_complex_fixed16 *inp_p,  
                                vspa_complex_fixed16 *out_p,  
                                structTXIQCompParams2 *cfg_p,  
                                unsigned int          num_lines);
```

with:

inp_p - Pointer to current batch within the input circular buffer (complex half-fixed, vector-aligned)

out_p - Output pointer (complex half-fixed, vector-aligned)

cfg_p - Configuration structure pointer

num_lines - Number of input/output DMEM lines (each line has 32 input/output samples)

Description:

This kernel works on an input buffer of “N” samples and produce an output buffer of N samples at 160 Msps and performs the following:

- IQ imbalance phase & amplitude compensation and fractional delay compensation
- DC offset compensation

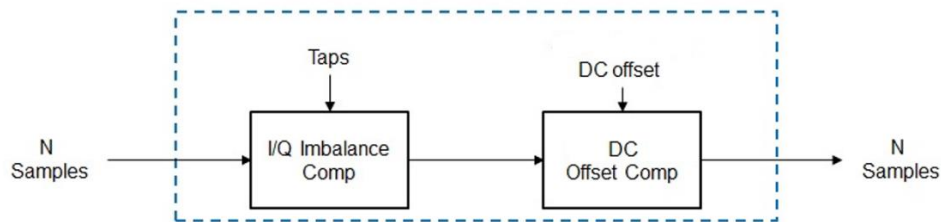


Fig 1: TX I/Q imbalance compensation + DC offset compensation

The I/Q imbalance compensation block is implemented as shown in Fig 2. The FIR filter h_1 is assumed to have 5 taps.

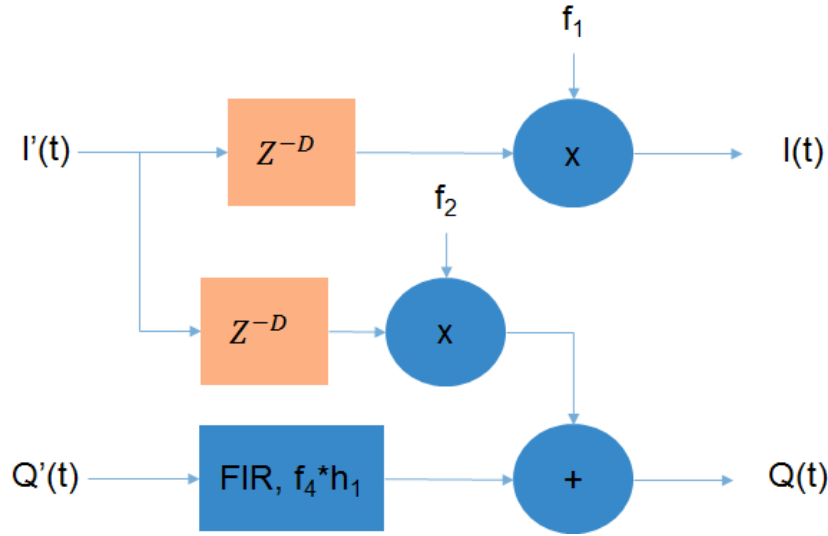


Fig 2: I/Q Imbalance Compensation Block

The I/Q imbalance compensation is based on the following impairment model.

$$\begin{bmatrix} I'(t) \\ Q'(t) \end{bmatrix} = \begin{bmatrix} \alpha & 0 \\ \sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} I(t) \\ Q(t - \tau) \end{bmatrix}$$

Here $\{I(t), Q(t)\}$ are real and imaginary parts of the original input signal, and $\{I'(t), Q'(t)\}$ are real and imaginary parts of the impaired signal. The symbol “ τ ” is a fractional (positive or negative) delay element. The I/Q imbalance compensation module shown in Fig applies the following correction to the impaired stream.

$$I(t) = f_1 * I'(t - D)$$

$$Q(t) = f_2 I'(t - D) + f_4 \sum_{k=0}^{L-1} h_1(k) * Q'(t - k)$$

Here D is a delay derived based on the delay, D_F , of FIR filter h_1 (of length L) adjusted for positive or negative “ τ ”. The filter h_1 is designed to shift the input signal by $(D_F + \delta)$. The parameters are derived as follows:

If $\tau > 0$:

$$\delta = 1 - \tau$$

$$D = D_F - 1$$

Elseif $\tau \leq 0$:

$$\delta = |\tau|$$

$$D = D_F$$

The parameters f_1, f_2, f_4 are derived as

$$f_1 = 1/\alpha$$

$$f_2 = -\tan(\varphi)/\alpha$$

$$f_4 = \sec(\varphi)$$



The DC compensation block is applied in the end:

$$I_{out}(t) = I_{DC} + I(t)$$
$$Q_{out}(t) = Q_{DC} + Q(t)$$

The details of the function API are as following.

Explicit arguments

inp_p: This is pointer to buffer containing input samples for the current channel at 160 Msps. The samples are expected to be complex and in half-fixed precision. The pointer can be any word (32-bit) aligned address and the buffers can be circular. N+32 valid input samples per channel are expected.

The input pointer needs to be set by the caller to point to the first input sample (not including the filter delay). The caller has to make sure there are at least “d” valid samples prior (in the circular buffer sense) to the first input sample, where d is the composite filter delay for the bandwidth.

out_p: This is pointer to buffer containing the final output samples for the current channels at 160 Msps. The samples are complex and in half-fixed precision. This pointer must be an integer multiple of the size of a DMEM line. N samples per channel are written per function call.

cfg_p: This is pointer to a struct of type `structTXIQCompParams2` that specifies configuration parameters for the signal conditioning chain corresponding to the channel being processed. This pointer must be an integer multiple of the size of a DMEM line. The caller must allocate and maintain one instance of this struct per channel. The structure is defined as:

```
typedef struct {  
    float          IQImb_ftaps[12];  
    unsigned int   IQImb_delay;  
    vspa_complex_float32 dcOffset;  
    vspa_complex_fixed16* inpCircBuffBase;  
    size_t         inpCircBuffSize;  
} structTXIQCompParams2;
```

Each field of the struct is defined below.

- **inpCircBuffBase**: This is pointer to the base of the circular buffer used for storing input samples for the channel.
- **inpCircBuffSize**: This is the size (in half words) of the input circular buffer.
- **dcOffset**: This parameter specifies the complex DC offset to be applied to the received signal for the channel at the input rate. This is a complex single precision floating point number. This value is applied in the chain with addition (and NOT subtraction). This value needs to be set by the caller.
- **IQImb_ftaps**: This is an array of single precision real coefficients of size 12 for I/Q imbalance compensation for the channel. The taps are arranged in memory as shown below (arranged in ascending order of array index from left to right):



For 5 taps case:

$[0, f_2, h_2(4), h'_1(4), h_2(3), h'_1(3), h_2(2), h'_1(2), h_2(1), h'_1(1), h_2(0), h'_1(0)]$

Here, for $k = 0, 1, 2, 3, 4$

$$h'_1(k) = f_4 h_1(k)$$

$$h_2(k) = \begin{cases} 0 & \text{if } k \neq D \\ f_1 & \text{if } k = D \end{cases}$$

The symbols $f_1, f_2, f_4, h_1(k)$ and D are coefficients and delay as defined earlier.

The FIR filter taps are applied to the imaginary input stream $q'(n)$ as shown below.

$$q(n) = h'_1(0)*q'(n) + h'_1(1)*q'(n-1) + h'_1(2)*q'(n-2) + h'_1(3)*q'(n-3) + h'_1(4)*q'(n-4)$$

This array needs to be populated by the caller and should entirely be contained within 1 DMEM line.

- [`IQImb\_delay`](#): Integer specifying the delay D for the integer path of the I/Q imbalance compensation block. Given D as described earlier, this value should be set to

$$IQImb_delay = 2*D + 1$$